

Encontrar el orden

- **Límite de tiempo:** 10 minutos
- **Entorno:** una GPU (≈ 16 GB VRAM), sin internet
- **Tamaño de la solución:** `solution.ipynb` ≤ 1 MB
- **Almacenamiento:** 5 GB

Problema

Se proporcionan diálogos hablados en inglés entre dos participantes, *Hablante A* y *Hablante B*. Cada diálogo está segmentado en turnos de habla, y cada turno contiene el habla de un único hablante. Cada turno se almacena como un archivo de audio `.wav` independiente, por lo que un diálogo completo está representado por un conjunto de archivos `.wav`, uno por cada turno.

Lamentablemente, los turnos se han mezclado aleatoriamente, por lo que la conversación ya no tiene sentido. En el nombre de archivo `chunk_{k}.wav`, k se refiere al k -ésimo fragmento del conjunto mezclado, no al k -ésimo turno del diálogo original.

!! Su tarea consiste en reconstruir el orden cronológico original de la conversación.



Dataset

Cada diálogo contiene n archivos de audio denominados `chunk_0.wav`, `chunk_1.wav`, ..., `chunk_{n-1}.wav`. Los fragmentos son turnos individuales. Los nombres de archivo corresponden únicamente al orden mezclado. No indican dónde corresponde un fragmento en la conversación original. Cada diálogo tiene 7-20 fragmentos, mono, 44.1 kHz (se puede remuestrear).

`prefix.json` contiene los índices de los nombres de archivo de los dos primeros fragmentos de cada diálogo. Esto identifica el verdadero comienzo del diálogo y elimina la ambigüedad entre leer la conversación hacia delante o hacia atrás.

Por ejemplo: 11: [7, 12] significa que los turnos primero y segundo del diálogo 11 son `chunk_7.wav` y `chunk_12.wav`, respectivamente.

Qué se proporciona

Se reciben **dos carpetas con formato idéntico**:

Carpeta	Diálogos	<code>answers.json</code> ?	Se utiliza para
<code>dataset/train/</code>	1,288	incluido	entrenar / ajustar su modelo
<code>dataset/test_public/</code>	100	incluido	ejecutar su pipeline y calcular localmente su propia puntuación

Durante la evaluación, su carpeta `dataset/test_public/` se sustituye de forma transparente por una `hidden evaluation set` (`test_leaderboard_a` para la tabla de clasificación pública y `test_leaderboard_b` para la tabla de clasificación final); estas tienen el mismo tamaño y formato que `dataset/test_public/`, pero sin `answers.json`.

Su notebook se ejecuta de nuevo con esos datos, y el archivo `answers.json` que genera se utiliza para calcular la puntuación. Los diálogos de prueba reservados proceden de la misma distribución que `train`, por lo que su puntuación local de `test_public` es una estimación fiel.

Estructura de directorios

```
dataset/train/
  prefix.json # {dialogue_id: [first_idx, second_idx]} filename index
  answers.json # {dialogue_id: P} ground-truth order (rank convention)
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav

dataset/test_public/
  prefix.json
  answers.json # present only in the development copy
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav
```

Salida

Para cada diálogo, determine el orden cronológico original de sus fragmentos de audio. Su predicción debe ser una permutación P de $\{0, 1, \dots, n-1\}$, donde $P[i]$ es la posición cronológica predicha de `chunk_i.wav` (0 = primero).

Su archivo de salida `answers.json` debe asociar cada ID de diálogo con su permutación predicha:

```
{
  "17": [2, 0, 1],
  "42": [0, 1],
  "108": [3, 1, 0, 4, 2, 5]
}
```

Ejemplo

Un diálogo tiene 3 fragmentos mezclados `chunk_0`, `chunk_1`, `chunk_2`:

fragmento mezclado	contenido hablado	posición verdadera (rango)
chunk_0.wav	"No worries — I'll send you the notes afterwards."	2 (último)
chunk_1.wav	"Hey, are you coming to the three o'clock meeting?"	0 (primero)
chunk_2.wav	"I can't — I've got a dentist appointment then."	1

El orden verdadero es **chunk_1** → **chunk_2** → **chunk_0**, por lo que $P = [2, 0, 1]$, y `prefix.json` contiene `[1, 2]`.

⚠ **P debe ser una permutación auténtica:** longitud n , indexada desde 0, cada valor exactamente una vez. Los valores duplicados, los valores ausentes o las entradas fuera de rango (p. ej., indexadas desde 1) obtienen una puntuación de 0 para ese diálogo, al igual que un diálogo ausente del archivo. Un archivo mal formado o que no sea JSON se rechaza.

Puntuación

La puntuación de esta tarea es la **exactitud del ordenamiento por pares**. Se comprueba cada par de fragmentos y se plantea: *¿cuál de los dos debe ir primero?* Un par es correcto si la predicción da la misma respuesta que la verdad de referencia. Para un diálogo con n fragmentos hay

$$M = n(n - 1)/2$$

pares; sea I el número de inversiones, es decir, pares ordenados de forma diferente a la verdad de referencia:

$$\text{score} = 1 - \frac{I}{M} \in [0, 1]$$

! **La puntuación final es el promedio de las puntuaciones por diálogo de todos los diálogos de la partición.**

Modelos permitidos

Solo se pueden utilizar los siguientes modelos preentrenados para resolver esta tarea, tanto durante el entrenamiento como durante la evaluación. Todos estos modelos ya están descargados y disponibles en el entorno. Se pueden consultar ejemplos de cómo utilizarlos en el notebook `baseline_solution.ipynb`. Tenga en cuenta que no se puede utilizar ningún otro modelo y que el programa no tiene acceso a internet.

- **Representaciones del habla: wav2vec 2.0.** El **codificador de Whisper** también puede utilizarse como extractor de características. [Ficha del modelo wav2vec](#)
- **Reconocimiento automático del habla (ASR): OpenAI Whisper** (cualquier tamaño). [Ficha del modelo Whisper](#)
- **Modelo de lenguaje: Qwen2.5-0.5B**, que puede utilizarse tanto zero-shot como ajustado con la partición `train` proporcionada. [Ficha del modelo Qwen](#) Tenga en cuenta que el límite de 10 minutos debe abarcar cualquier entrenamiento o ajuste que se realice durante la evaluación, además de la inferencia sobre el conjunto de evaluación.

Cómo enviar

- Abra `solution.ipynb` y ejecute todas las celdas. Confirme que genera `answers.json` en el directorio de trabajo con una permutación para cada diálogo de `dataset/test_public/` (100 diálogos). Durante la evaluación, el notebook se vuelve a ejecutar en el conjunto de prueba oculto y se puntúa el `answers.json` que genera allí.
- Mejore la solución si lo desea, o no lo haga; el baseline por sí solo valida el pipeline.
- Abra la pestaña Git de la barra lateral izquierda de JupyterLab.
- Añada al área de preparación (**Stage**) `solution.ipynb` (el icono + situado junto al archivo).
- Introduzca un mensaje de commit y haga clic en **Commit**.
- Haga clic en el icono de la nube con una flecha hacia arriba para hacer push.
- Vuelva a esta página del concurso y haga clic en **Submit**.

Envíe exactamente un archivo, denominado `solution.ipynb`.